NLA PROJECT 1.

Group 7

AI acknowledgment

AI was used for assistance with LaTeX formatting in overleaf. Concretely, we used it to present our MATLAB code as nice code boxes that distinguish themselves from ordinary text and to help us place our graphs/figures into the document in the right position.

Context

We consider a rectangular plate $R = [0, 2] \times [0, 3]$ and denote the temperature at time t at position $(x, y) \in R$ by $u(t, x, y)$.

We assume that u satisfies:

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + s(x, y)$$

such that $s(x, y) = \begin{cases} 15 & x \in (0.25, 1.25) \times (0.75, 1.25) \\ 0 & \text{otherwise} \end{cases}$ We fix $u = 0$ at the boundary of the domain. we choose N_x gridpoints in the x direction and N_y gridpoints on the y direction.

Let $x_i = (i-1)\Delta x$ such that $i = 1, ..., N_x$ and similarly t $y_j = (j-1)\Delta y$ such that $j = 1, ..., N_y$ and furthermore let $\Delta x = \dfrac{2}{N_x - 1}$ and $\Delta y = \dfrac{3}{N_y - 1}$

Grid point (x_i, y_j) has label $k = (j-1)N_x + i$. We further defined $u_k(t)$ as the numerical approximation of $u(t, x_i, y_j)$,

Let $\mathbf{u}(t) = (u_1(t), ..., u_{NxNy}(t))$

Question 1

(a) We need to find the discretization of the center difference of $\dfrac{\partial^2 u}{\partial x^2}$. It holds that the second order partial derivative at a point (x_i, y_j) can be approximated by

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u(t, x_{i-1}, y_j) - 2u(t, x_i, y_j) + u(t, x_{i+1}, y_j)}{(\Delta x)^2}$$

Using the grid label definition, this can be rewritten as

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{k-1} - 2u_k + u_{k+1}}{(\Delta x)^2}$$

Question 2

(b) We need the discretization of the centre difference of $\dfrac{\partial^2 u}{\partial y^2}$. It holds that the second order partial derivative at a point (x_i, y_j) can be approximated by

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u(t, x_i, y_{j-1}) - 2u(t, x_i, y_j) + u(t, x_u, y_{j+1})}{(\Delta y)^2}$$

Using the grid label definition, this can be rewritten as

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{k-N_x} - 2u_k + u_{k+N_x}}{(\Delta y)^2}.$$

Question 3

(c) We need to write $\frac{d\mathbf{u}}{dt} = A\mathbf{u} + \mathbf{b}$. and find the matrix A and the vector $\mathbf{b}$ using our discretized version.

We can write $\frac{d\mathbf{u}}{dt}$ as

$$\frac{d\mathbf{u}}{dt} = \begin{pmatrix} \frac{du_1}{dt} \\ \frac{du_2}{dt} \\ \vdots \\ \frac{du_{N_x N_y}}{dt} \end{pmatrix}$$

hence we need to find an expression for each $\frac{du_k}{dt}$.

Case 1: the point lies in the boundary of the rectangle:

For points in the boundary, meaning $i = 1$ or $i = N_x$ or $j = 1$ or $j = N_y$ since we established that $u = 0$ then we have that $\frac{du_k}{dt} = 0$ such that this holds for any k where $i = 1$, $i = N_x$, $j = 1$, or $j = N_y$

Case 2: The points lie inside the boundary:
For points inside the boundary (meaning that $i \neq 1, i \neq N_x, j \neq 1, j \neq N_y$), it holds that

$$\frac{du_k}{dt} = \frac{u_{k-1} - 2u_k + u_{k+1}}{(\Delta x)^2} + \frac{u_{k-N_x} - 2u_k + u_{k+N_x}}{(\Delta y)^2} + s(x_i, y_j)$$

$$\iff \frac{du_k}{dt} = \frac{u_{k-N_x}}{(\Delta y)^2} + \frac{u_{k-1}}{(\Delta x)^2} - 2u_k \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} \right) + \frac{u_{k+1}}{(\Delta x)^2} + \frac{u_{k+N_x}}{(\Delta y)^2} + s(x_i, y_j).$$

We will now construct the matrix. Based on the above, we can write the system as

$$\begin{pmatrix} \frac{du_1}{dt} \\ \frac{du_2}{dt} \\ \vdots \\ \frac{du_k}{dt} \\ \vdots \\ \frac{du_{N_x N_y}}{dt} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ \frac{u_{k-N_x}}{(\Delta y)^2} + \frac{u_{k-1}}{(\Delta x)^2} - 2(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2})u_k + \frac{u_{k+1}}{(\Delta x)^2} + \frac{u_{k+N_x}}{(\Delta y)^2} \\ \vdots \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ \vdots \\ s(x_i, y_j) \\ \vdots \\ 0 \end{pmatrix}$$

For a point u_k which is not in the boundary, the expression depends on 5 grid points, which are the grid points with indices $k, k+1, k-1, k+N_x, k-N_x$, hence in the matrix A in row k, only the columns which match these indices must be non zero.It hence holds that for every row k we have that

$$A_{k,k} = -2(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}), A_{k,k-1} = \frac{1}{\Delta x^2}, A_{k,k+1} = \frac{1}{\Delta x^2}, A_{k,k-N_x} = \frac{1}{\Delta y^2}, A_{k,k+N_x} = \frac{1}{\Delta y^2}$$

and $A_{k,l} = 0$ otherwise, such that u_k is not in the boundary. If u_k is in the boundary, then the whole row k is 0.

More precisely, if $k = (j-1)N_x + i$ we can properly define A as

$$A_{kl} = \begin{cases} 0 & \text{if } i = 1, i = N_x, j = 1 \text{ or } j = N_y \\ 0 & \text{if } l \notin \{k, k-1, k+1, k-N_x, k+N_x\} \\ -2(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}) & \text{if } l = k \\ \frac{1}{(\Delta x)^2} & \text{if } l = k-1 \text{ or } l = k+1 \\ \frac{1}{(\delta y^2)} & \text{if } l = k+N_x \text{ or } l = k-N_x \end{cases}$$

We further have that $\mathbf{b} = \begin{pmatrix} b_1 \\ \vdots \\ b_k \\ \vdots \\ b_M \end{pmatrix}$ with $b_k = \begin{cases} 0 & \text{if } u_k \text{ is in the boundary,} \\ s(x_i, y_j) & \text{if } u_k \text{ is in the interior..} \end{cases}$

Applying the above definition to $N_x = 4$ and $N_y =$ we obtain that $\Delta x = \frac{2}{3}, \Delta y = \frac{3}{5}$ so

$$\frac{1}{\Delta x^2} = \frac{9}{4}, \qquad \frac{1}{\Delta y^2} = \frac{25}{9},$$

which we can use to construct the matrix A. For the vector b, the only point which satisfies $0.25 < x < 1.25$ and $0.75 < y < 1.25$ is $(\frac{2}{3}, \frac{6}{5})$ which has label $k = (3-1)4 + 2 = 10$, hence the vector $b \in \mathbb{R}^{24}$ as full 0 entries except for the 10th entry.

Question 4

By the implicit Euler method, we have:

$$\begin{aligned} & \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} = A\mathbf{u}^{n+1} + \mathbf{b} \\ \iff & \mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t \cdot (A\mathbf{u}^{n+1} + \mathbf{b}) \\ \iff & \mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t A\mathbf{u}^{n+1} + \Delta t\mathbf{b} \\ \iff & \mathbf{u}^{n+1} - \Delta t A\mathbf{u}^{n+1} = \Delta t\mathbf{b} + \mathbf{u}^n \\ \iff & (I - \Delta t A)\mathbf{u}^{n+1} = \Delta t\mathbf{b} + \mathbf{u}^n \end{aligned}$$

Hence, we have $B\mathbf{u}^{n+1} = \mathbf{c}$ with

$$B = (I - \Delta t A), \qquad \mathbf{c} = \Delta t\mathbf{b} + \mathbf{u}^n.$$

where I is the identity matrix.

Question 5

We complete the given code as follows.

```
close all
clear all

% Dimensions of the plate
xmin = 0; xmax = 2; ymin = 0; ymax = 3;

```

```
% Grid
Nx = 40; Ny = 60;
dltx = (xmax-xmin) / (Nx-1); dlty = (ymax-ymin) / (Ny-1);
[x, y] = ndgrid( xmin + (0:Nx-1)*dltx, ymin + (0:Ny-1)*dlty);

% Source term
s = 15 * (x>0.25 & x<1.25 & y>0.75 & y<1.25);
figure; pcolor(x,y,s); xlabel('x'); ylabel('y'); title('Source')
colorbar; shading interp; colormap hot; axis equal; axis tight

% Build linear system
A = zeros(Nx*Ny,Nx*Ny);
b = zeros(Nx*Ny,1);

% Internal grid points (points that have a north, east, south and west neighbor)
for i = 2:Nx-1
    for j = 2:Ny-1
        C = (j-1) * Nx + i;   % Label of grid point (i,j)
        E = C + 1;            % East neighbor
        W = C - 1;            % West
        N = C + Nx;           % North
        S = C - Nx;           % South

        % Build the matrix
        A(C,C) = -2 / dltx^2 - 2 / dlty^2;
        A(C,E) = 1 / dltx^2;
        A(C,W) = 1 / dltx^2;
        A(C,N) = 1 / dlty^2;
        A(C,S) = 1 / dlty^2;

        % Right-hand side
        b(C) = s(i,j);
    end
end

% Boundary Conditions
% Top
for i = 1:Nx
    j = Ny; C = (j-1) * Nx + i; A(C,C) = 1; b(C) = 0;
end
% Bottom
for i = 1:Nx
    j = 1; C = (j-1) * Nx + i; A(C,C) = 1; b(C) = 0;
end
% Left
for j = 2:Ny-1
    i = 1; C = (j-1) * Nx + i; A(C,C) = 1; b(C) = 0;
end
% Right
for j = 2:Ny-1
    i = Nx; C = (j-1) * Nx + i; A(C,C) = 1; b(C) = 0;
end

% Time stepping
u = zeros(Nx*Ny,1);   % Initial solution
dltt = 0.1;           % Time step
Nt = 30;              % Number of time steps
f = figure(2); hold on
for i = 1:Nt
    t = i*dltt;
    % Find the vector u at the next time point
    u = (eye(Nx*Ny) - A*dltt) \ (b*dltt + u);
    % Plot solution
    um = reshape(u,Nx,Ny);   % Reshape vector to matrix for plotting
    figure(2)
    pcolor(x,y,um); shading interp; colormap hot; colorbar;
    xlabel('x'); ylabel('y'); title(sprintf('t = %g', t))
    clim([0 3])   % Fix the colors
    axis equal; axis tight
    M(i) = getframe(f);   % Store the figure to make movie
end
```

```
figure(2)
movie(f, M, 5)    % Play the movie five times
```

When we run the code, the first figure shows the source term s, which gives the position and size of the CPU (Figure 1, left). The second figure shows the temperature u as it evolves in time. At $t = 0$ the plate and the CPU have temperature 0, as prescribed by the initial condition. The temperature then rises quickly, reaching a value of around 2.5 at approximately $t = 1.5$. After that, the solution reaches an equilibrium state. Because the CPU is placed in the lower left part of the plate, the heat spreads out from there, and even at the final time $t = 3$ it does not reach the top part of the plate. The heat escapes through the bottom and left boundaries, since these are fixed at $u = 0$ and are closest to the CPU. This can be seen from the fact that the heat does not spread out as much in these directions as it does towards the top.

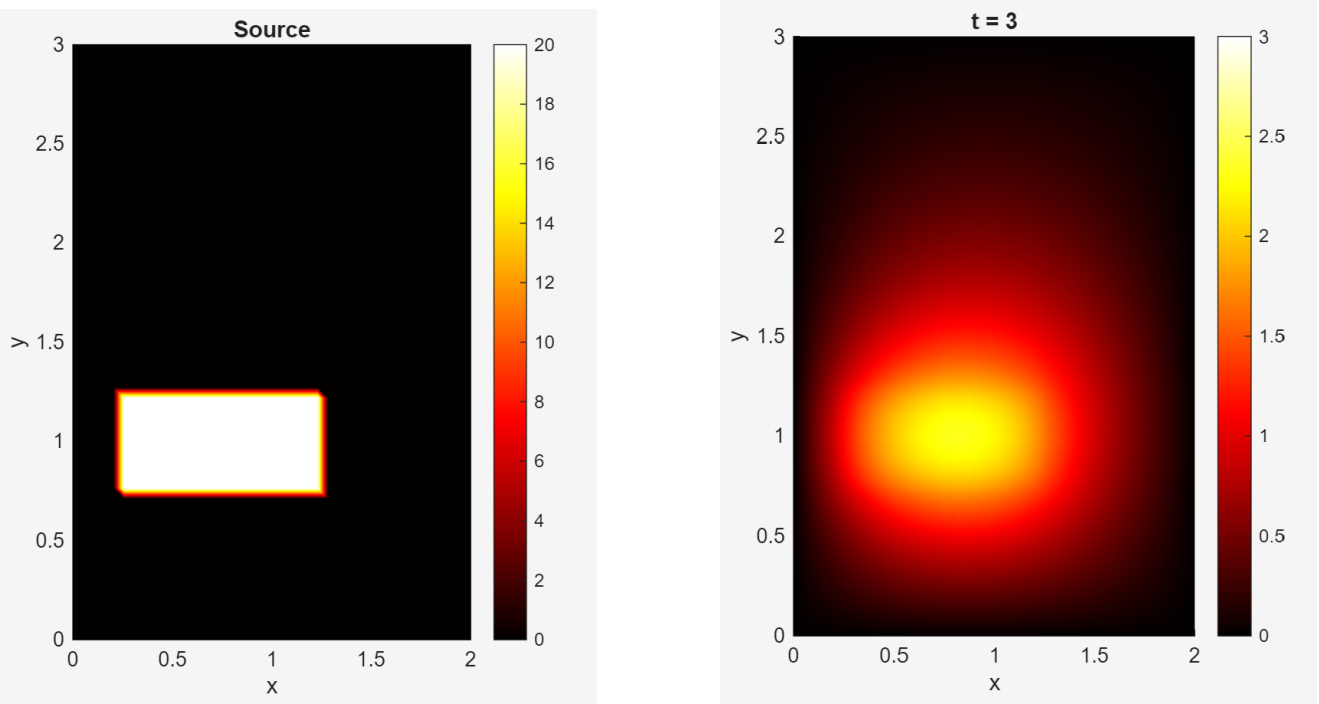


Figure 1: Source term s (left) and numerical solution u at $t = 3$ (right).

Question 6

We use the maximum temperature on the plate as the main metric for comparison, and we take the sizes of the CPU and the heat sink in mm. To display the maximum temperature in the figures, we modified the title in the time loop:

```
title(sprintf('t = %g, Max Temp = %.2f °C', t, 20 + (max(u) * 15)))
```

Here 20 is the room temperature (in °C), and 15 is a scaling factor we chose to convert the values of $\max(u)$ to degrees.

Configuration (a)

As our first configuration, we kept the same CPU size as in the given experiment, but moved the CPU to the centre of the plate. Since it is far from all the boundaries that can cool it down, it reaches a higher temperature than in the original set-up. This configuration is not as efficient: the heat escapes fastest through the left and right boundaries, but even there it takes a while, and the CPU becomes too hot. The maximum temperature recorded was 57.67°C.

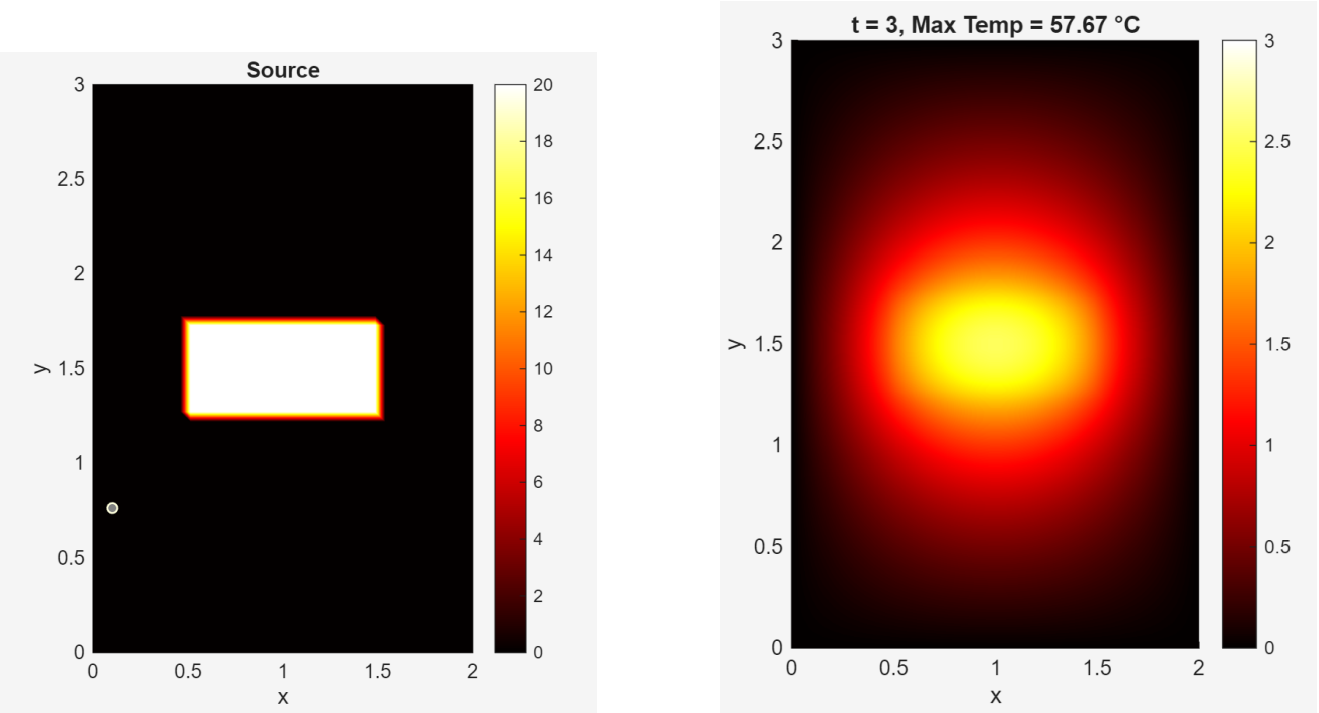
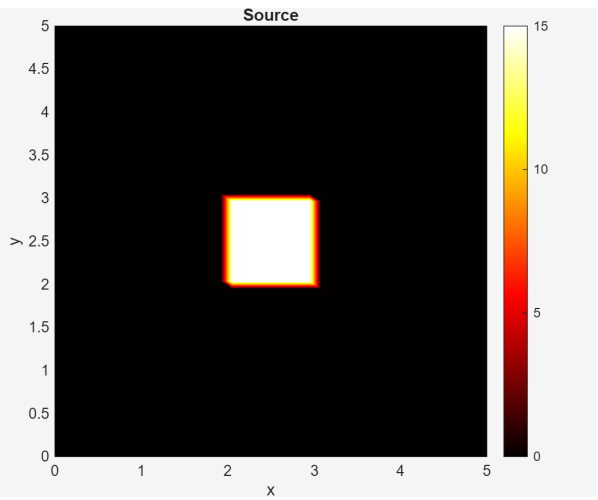


Figure 2: Configuration (a): source term (left) and solution at $t = 3$ (right).

Configuration (b)

Here we made the heat sink much bigger, 5×5 mm, kept the CPU in the centre, and made the CPU slightly bigger, 1×1 mm. Because of the large distance between the CPU and the boundaries where the heat can escape, the temperature gets a lot higher, nearly double that of the first configuration. Despite the larger heat sink, the CPU only gets hotter, since the heat has a lot of room to spread into. The maximum temperature is 91.12°C.



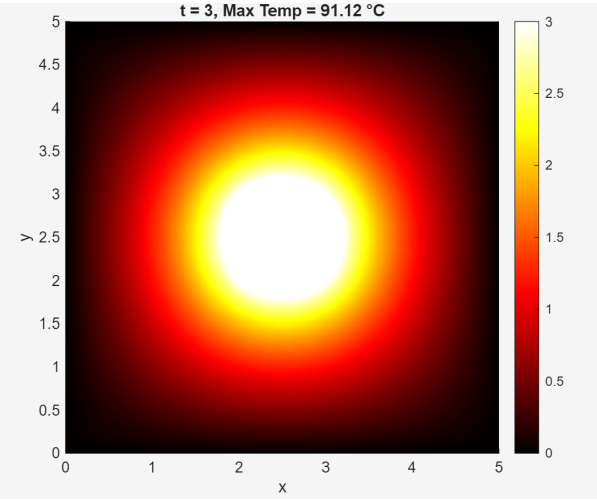
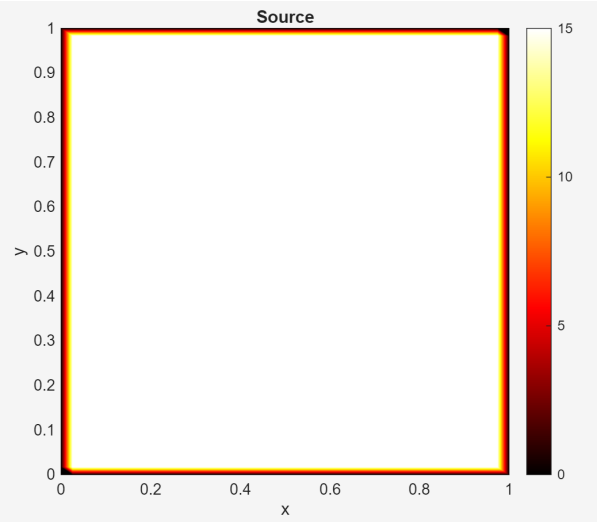


Figure 3: Configuration (b): source term (left) and solution at $t = 3$ (right).

Configuration (c)

As our last configuration, we made the heat sink the same size as the CPU (1×1). The maximum temperature is 36.56°C, the lowest of all configurations. This is because the edges of the plate are fixed at room temperature, so the edges of the CPU are now cooled directly. However, this means there is effectively no heat sink, and it assumes perfect cooling at the CPU's edges, which is not realistic. We therefore do not consider (c) a valid design, but rather an illustration of a limitation of the model.



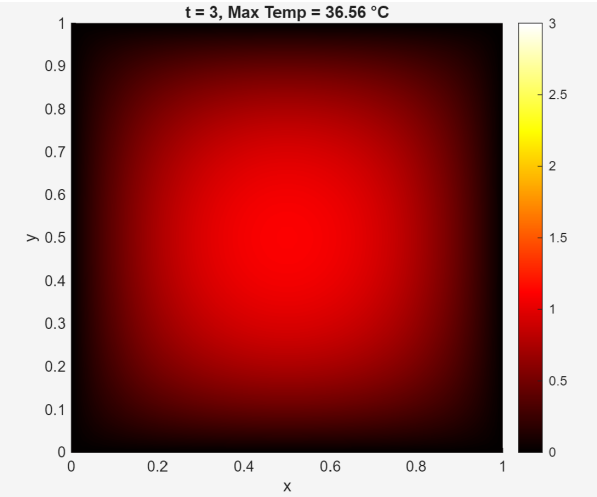


Figure 4: Configuration (c): source term (left) and solution at $t = 3$ (right).

Conclusion

Of the realistic configurations, (a) has the lowest maximum temperature (57.67°C, compared to 91.12°C for (b)), so we recommend configuration (a). In this model heat can only escape through the edges, so the closer the CPU is to them, the cooler it stays. Configurations (b) and (c) suggest that a smaller heat sink is always better, but this is only because the model has no cooling through the surface of the plate, so the model cannot properly judge the size of the heat sink.

Question 7

Firstly, in a more realistic setup, the model should be modeled as a 3D structure, not a 2D plate, since heat moves through the thickness of the CPU and thus changes depending on the height of the point it is at.

Second, the boundary conditions are not realistic either. The model assumes that the edges remove heat perfectly, but in a real-world scenario this is impossible, as heat is lost gradually to the surrounding air due to physical processes like convection, making this unrealistic. Morever heat is lost through the entire surface, not only through the edges which is exactly why a larger heat sink cools better in practice, whereas in our model configuration (b) and (c) found that a larger plate only made the CPU hotter.

Finally, another key point is that this model treats the whole plate as one uniform structure, while in most computers these plates are made of various materials like silicon, aluminum, or copper, with different thermal conductivities, densities, and heat capacities. This means that the model is too simplistic in this sense, as the presence of different materials in different parts of the CPU affects how heat travels through these specific regions, meaning that these variables should be taken into account.